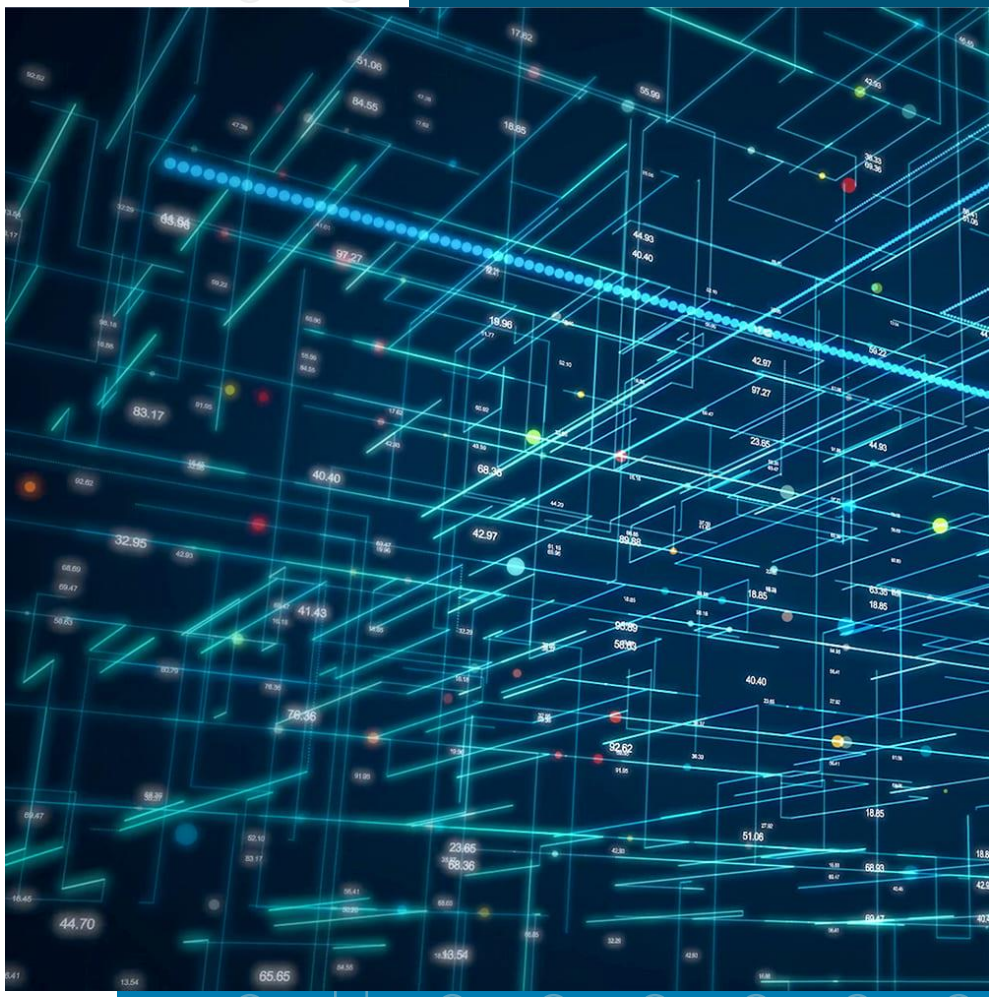
SIOP ANNUAL CONFERENCE

NEW ORLEANS • April 29–May 2, 2026

Designing and Building AI Agents for High-Stakes IO Applications

A Master Tutorial

30 April 2026 | SIOP 2026 | New Orleans, LA



SHL Codility rhr ALIVE INSTITUTE

© 2026 SHL and its affiliates. All rights reserved.

Presenters

Karim Badr

Senior Research Scientist

SHL

James Meaden

Independent Consultant / Founder

Codility

ALIVE INSTITUTE

Cameron Beazley

Science Director

SHL

Andrew Samo

Senior Research Scientist

rhr

© 2026 SHL and its affiliates. All rights reserved.

Agenda

1. **Setting the Scene**: What is Agentic AI?
2. The Software Development Life Cycle (SDLC) x Psychometrics **Framework**
3. Agentic AI **No-Code** Showcase using *n8n*
4. **Coding** Showcase: *Agentic AI with Python*
5. Q&A + Resources for your learning

© 2026 SHL and its affiliates. All rights reserved.

Agenda

1. **Setting the Scene**: What is Agentic AI?
2. The Software Development Life Cycle (SDLC) x Psychometrics **Framework**
3. Agentic AI **No-Code** Showcase using *n8n*
4. **Coding** Showcase: *Agentic AI with Python*
5. Q&A + Resources for your learning

© 2026 SHL and its affiliates. All rights reserved.

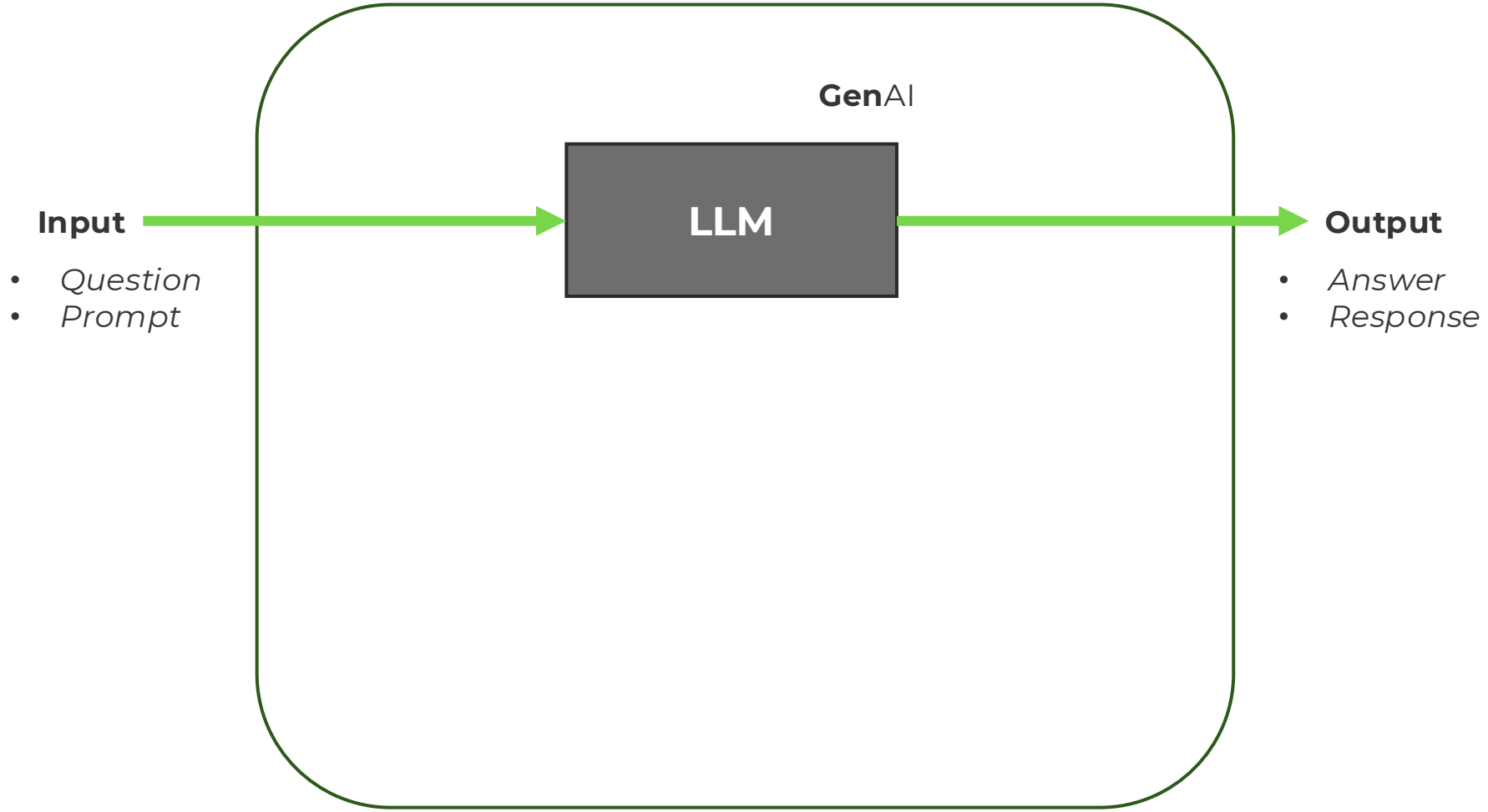
Setting the Scene: What is Agentic AI?

AI **Chatbot** v.s. AI **Agent**

© 2026 SHL and its affiliates. All rights reserved.

Setting the Scene: What is Agentic AI?

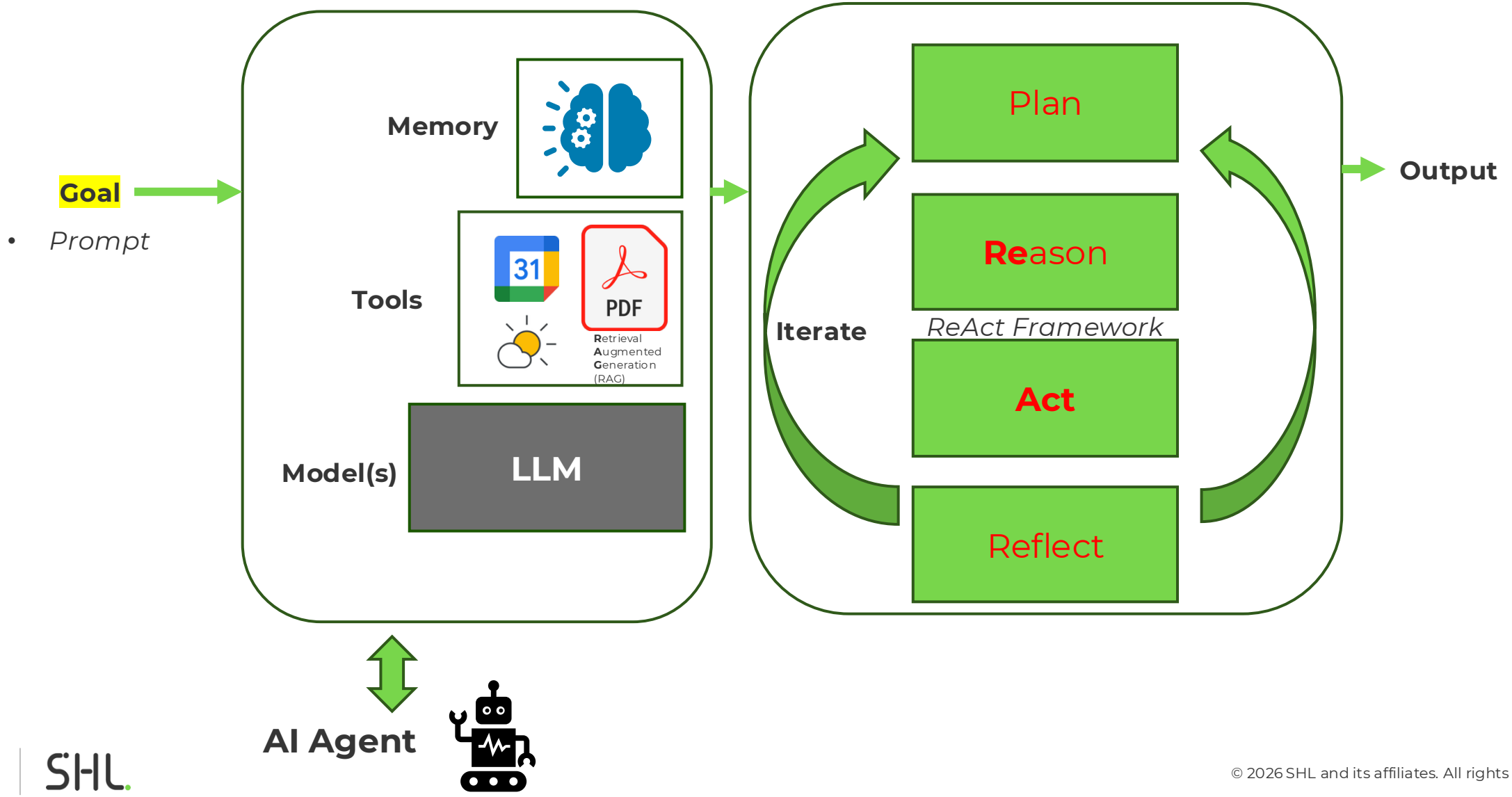
AI **Chatbot**



© 2026 SHL and its affiliates. All rights reserved.

Setting the Scene: What is Agentic AI?

AI **Agent**



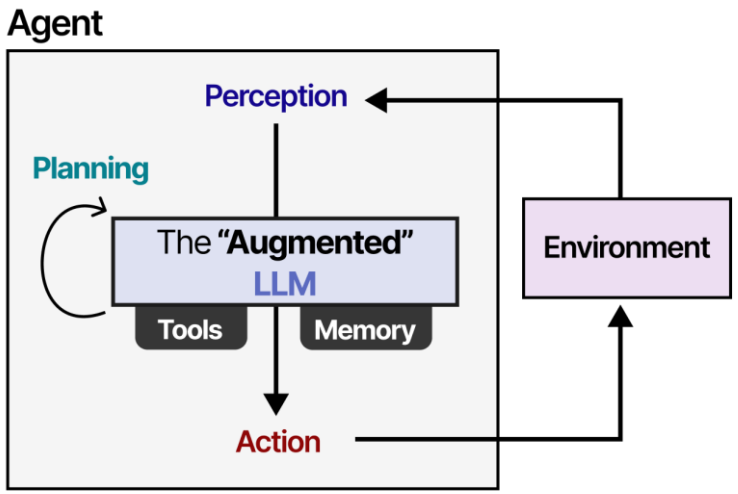
SHL.

© 2026 SHL and its affiliates. All rights reserved.

Setting the Scene: What is Agentic AI?

*An Agent is a system that leverages an AI model to **interact** with its environment in order to achieve a user-defined **objective**. It combines **reasoning**, **planning**, and the **execution** of actions (often via external tools) to fulfil tasks.*

AI Chatbot	AI Agent
Chats	**Acts**
Reactive	**Proactive**
Responds to what you type	Plans what to do and executes
Follows preset flows	Plans next steps
Serves canned answers	Learns from memory



© 2026 SHL and its affiliates. All rights reserved.

Agenda

1. **Setting the Scene**: What is Agentic AI?
2. The Software Development Life Cycle (SDLC) x Psychometrics **Framework**
3. Agentic AI **No-Code** Showcase using *n8n*
4. **Coding** Showcase: *Agentic AI with Python*
5. Q&A + Resources for your learning

© 2026 SHL and its affiliates. All rights reserved.

Why Do We Need a Framework?

The gap between building agents and building them *well*

The Problem

Agentic AI tools being deployed in HR workflows — hiring, assessment, development — **without the rigor those decisions demand.**

- Ad-hoc builds with no construct specification
- No reliability or validity evidence collected
- Fairness untested; bias risks unexamined
- No audit trail when decisions are challenged

The Opportunity

I/O psychologists already have the expertise to close this gap — **we just haven't applied it to agents yet.**

- Construct validation is what we do
- Fairness analysis is embedded in our practice
- We understand high-stakes HR decisions
- SIOP Principles and UGESP already define what rigor looks like

Three Fields, One Framework

Integrating the best of three disciplines

Software Development Lifecycle

Structured phases from planning to maintenance — ensuring agents are built with production-grade rigor.

- Requirements engineering
- Version control and testing
- Iterative deployment

Psychometric Assessment Standards

Construct clarity, reliability, validity, and fairness — anchored to SIOP Principles, UGESP, and the Standards (AERA/APA/NCME).

- Construct specification
- Multi-layer validity evidence
- Fairness and adverse impact analysis

AI Evaluation Methods

Systematic technical methods for testing, benchmarking, and validating AI outputs — the operational layer that makes psychometric standards actionable in practice.

- LLM-as-judge & automated eval harnesses
- Red-teaming & adversarial probing
- Benchmark datasets & regression testing

Who Is This Framework For?

Builders and evaluators — democratisation makes both roles urgent

For Builders

Use the 8 steps as a development checklist — what to build, what to test, what evidence to document before moving on.

For Evaluators

Use the 8 steps as a procurement audit. When a vendor pitches you an AI agent, run it through Steps 1–5. Can they answer your questions?

THE DEMOCRATISATION REALITY

No-code tools like n8n, Make, and Zapier AI mean anyone can now build an AI agent.

This doesn't reduce the need for validation — **it amplifies it.** A recruiter building an agent to screen CVs needs the same validity evidence as a fully engineered system, because the decisions are the same.

The tiered approach on the next slide helps calibrate how much rigour each context demands.

The Framework at a Glance

Eight integrated steps — from governance to iteration

01

Governance & Purpose

Define use, stakes, construct, governance

02

Requirements

Functional + psychometric blueprint

03

Design

Architecture, prompts, evaluation scaffolds

04

Build

Implement, version, evaluation harness

05

Validation

Content, reliability, validity, fairness, safety

06

Deployment

Staged rollout, HITL, user training

07

Monitoring

Drift, fairness, adoption, re-validation triggers

08

Iteration

Re-validate, refine, document, communicate

Tiered Validation Scale

Stakes classification calibrates required rigour — not every agent needs the same standard

TIER 1
High-Stakes

Outputs directly inform consequential HR decisions

Hiring, promotion, performance assessment, termination. Treat as equivalent to a selection instrument. **Full 8-step validation. UGESP and SIOP Principles apply in full.**

Full Validation
All 5 validity layers + audit trail

TIER 2
Medium-Stakes

Advisory or analytic outputs

Talent planning, competency mapping, development recommendations. Human-owned decisions. **Abbreviated validation: content + reliability + fairness check.**

Abbreviated
Content + reliability + fairness

TIER 3
Low-Stakes

Productivity or drafting aids

Email drafting, report summarisation, search assistance. Outputs not used directly in decisions. **Functionality testing + lightweight monitoring sufficient.**

Lightweight
Functional testing + monitoring

01 Governance & Purpose Definition

The foundational layer — before design or build work begins

KEY ACTIVITIES

Construct specification: Define precisely what the agent measures, produces, or supports. Ambiguous constructs produce invalid agents.

Stakes classification: Apply the Tiered Validation Scale to determine required rigour level.

When NOT to use an agent: Is a rules-based system, simpler LLM call, or human-only process the better choice?

Human-in-the-loop design: Determine required oversight level — this decision flows into every subsequent design choice.

Applicable standards: Identify which standards govern this use: UGESP, SIOP Principles, EU AI Act.

Documentation baseline: Establish the audit trail standard that persists through all 8 steps.

IN PRACTICE: AI JOB ANALYSIS ASSISTANT

Clarify that the agent synthesises job descriptions into competency models to support HR decision-making. Classify as Tier 2 (advisory, medium-stakes).

Determine HITL: recruiters review and approve all outputs before use.

Applicable standards: SIOP Principles, UGESP adverse impact provisions.

02 Requirements

Operationalise the construct — the psychometric blueprint for the system

KEY ACTIVITIES

Functional requirements: What tasks does the agent perform? What inputs in, what outputs out?

Non-functional requirements: Performance thresholds (accuracy, consistency), transparency, accessibility, privacy.

Psychometric blueprint: Construct definition → observable indicators → scoring rationale → mapping to validated frameworks (O*NET, competency models).

AI Eval strategy upfront: Specify eval metrics, benchmark datasets, and quantitative success thresholds before a single prompt is written — the AI Evals equivalent of test-driven development. Define what 'good enough' looks like before you build.

SME identification: Who provides the expert criterion for validation? SME judgements become the gold standard that automated evals are calibrated against.

IN PRACTICE: AI JOB ANALYSIS ASSISTANT

Functional: extract and categorise competencies from job description text. Non-functional: accuracy >80% vs. SME benchmark, transparent citations.

Blueprint: map extracted competencies to the organisation's validated competency model.

SMEs: senior HR analysts who perform this manually today.

03 Design

Architect the agent — decisions here have downstream validity consequences

KEY ACTIVITIES

Agent architecture: Workflow, tool selection, memory design (short-term vs. RAG), reasoning approach.

Prompt design as construct operationalisation: Prompts are the operational definition of the construct. Inconsistent prompts = construct-irrelevant variance — same threat as poorly written test items.

Model selection: Higher stakes require more stable, interpretable, well-documented models.

Evaluation-driven design: Write LLM-as-judge rubrics and automated test cases before writing prompts — the AI Evals equivalent of TDD. Design the eval harness architecture alongside the agent architecture.

Reasoning transparency: For Tier 1 applications, design chain-of-thought outputs that can be audited. Interpretable reasoning is itself an eval target.

IN PRACTICE: AI JOB ANALYSIS ASSISTANT

Choose retrieval-augmented generation with structured competency databases. Design structured LLM-as-judge rubrics for assessing competency coverage and clarity before any code is written. Select a model with documented performance characteristics and version stability.

04 Build

Implement with production rigour — not a prototype mindset

KEY ACTIVITIES

Version control everything: Prompts, code, and data are all versioned. Any change may require re-validation.

Evaluation harness from the start: LLM-as-judge + deterministic checks running continuously against the benchmark dataset. Track eval scores across every prompt and model version — the signal that tells you whether a change helped or hurt.

Logging infrastructure: All inputs, outputs, and reasoning traces are logged. Non-negotiable for Tier 1.

SME content review during build: Domain experts review outputs on representative inputs — early expert review catches construct misalignment before it propagates.

System documentation: System card, prompt library, data lineage, and running change log form the technical report equivalent.

IN PRACTICE: AI JOB ANALYSIS ASSISTANT

Code connectors to HRIS databases; implement structured prompt libraries; build automated test harness with nightly comparisons against gold-standard expert competency mappings; conduct weekly SME review sessions. Log all outputs in a structured database.

05 Validation

Where psychometric standards and AI Eval methods converge

Content Validity

SME review panels assess output domain accuracy and construct representativeness. Coverage analysis across full content blueprint.

↳ AI Evals: LLM-as-judge panels; expert-graded benchmark sets

Reliability

Temporal consistency (same input → same output); inter-rater equivalent (agent vs. expert agreement); internal coherence.

↳ AI Evals: Automated re-run testing; inter-evaluator agreement (κ)

Construct / Criterion Validity

Convergent evidence; criterion prediction; discriminant evidence vs. adjacent constructs.

↳ AI Evals: Held-out benchmark correlation; gold-standard comparison

Fairness

Disparate impact analog; differential output quality by demographic group; intersectional analysis for Tier 1.

↳ AI Evals: Counterfactual probing; bias benchmarks

Safety & Usability

Red-teaming; appropriate reliance testing (automation bias check); interpretability review.

↳ AI Evals: Adversarial probing; jailbreak resistance testing

IN PRACTICE: AI JOB ANALYSIS ASSISTANT

Content:

SME panel of 5 senior HR analysts rates agent outputs on coverage, accuracy, and clarity against the competency blueprint.

Reliability:

Re-run 50 job descriptions 3× each; calculate ICC with expert panel. Target ICC ≥ 0.75.

Fairness:

Analyse whether output language systematically differs for identically-skilled roles with gendered job titles.

06 Deployment

Controlled release — not a switch that gets flipped

KEY ACTIVITIES

Staged rollout: Pilot with a representative subset before broad deployment. Feature flags enable rapid rollback.

Activate human-in-the-loop: Implement the oversight structure from Step 1. Deployment conditions define the boundary within which validity evidence holds.

User training: Users must understand what the agent does, its limitations, and when to escalate. *Appropriate reliance is a training outcome.*

Boundary condition documentation: Validity evidence applies only under the specific conditions in which it was collected.

Incident response protocol: Define what constitutes an incident, who is notified, and the response procedure — before the first incident.

IN PRACTICE: AI JOB ANALYSIS ASSISTANT

Release to 10 recruiters for 60 days. All outputs reviewed by a senior HR analyst before reaching hiring managers. Provide structured training on interpreting competency outputs. Document that validity evidence applies only to tech-sector roles (the pilot population).

07 Monitoring

Continuous performance tracking — with explicit re-validation triggers

KEY ACTIVITIES

Production eval pipelines: The same AI Eval harness from Steps 4–5 runs continuously on live samples — automated drift detection before users report problems.

Drift detection: Statistical comparison of live eval scores against Step 5 baseline. Changes in output quality are often invisible to users until they've compounded.

Ongoing fairness monitoring: Continuous counterfactual probing and bias benchmark checks; output quality parity across demographic groups.

User behaviour analytics: Override rates, escalation rates — indicators of appropriate or inappropriate reliance.

Re-validation triggers (define upfront): Provider model update | Eval score drops below threshold | Drift breach | Incident threshold | Material population change.

IN PRACTICE: AI JOB ANALYSIS ASSISTANT

Dashboard tracks weekly AI Eval scores against Step 5 baseline, error report rates, and subgroup output parity. Alert when eval score drops >10% from baseline.

Policy: any provider model update triggers a 30-day re-validation sprint. Quarterly fairness review by the I/O team.

08 Iteration

Structured re-validation cycles — closing the evidence loop

KEY ACTIVITIES

Periodic re-validation: Annually for Tier 1; triggered by defined criteria for Tier 2 and 3.

Systematic evidence updates: New SME reviews, refreshed benchmark datasets, updated fairness analyses — not just prompt tweaks.

Change documentation: Every change to prompt, model, data, or rubric is logged with rationale — maintaining the audit trail as a living document.

Stakeholder communication: Communicate changes and updated evidence to governance owners, users, and affected groups.

Tier creep check: Re-assess whether classification still reflects actual use. Agents drift toward higher stakes as organisations rely on them more.

IN PRACTICE: AI JOB ANALYSIS ASSISTANT

Incorporate quarterly recruiter and SME feedback. Refresh benchmark dataset every 6 months. Full re-validation annually. Document all prompt version changes with rationale. Annual governance review: 'Is this still advisory (Tier 2), or has it effectively become Tier 1?'

Where I/O Expertise Adds Unique Value

You don't need to own every step — but these are the ones where I/O training is irreplaceable

01

Governance & Purpose

I/O LEADS

Construct specification and stakes classification are I/O work.

02

Requirements

I/O LEADS

Psychometric blueprint design — what to measure, how to score it, what counts as quality — is the core I/O contribution to requirements.

03

Design

I/O CONTRIBUTES

Prompt design as construct operationalisation. Evaluation rubric design. I/O and engineering collaborate closely here.

04

Build

ENGINEERING LEADS

Implementation is primarily engineering. I/O role: SME content review during build and sign-off on the evaluation harness design.

05

Validation

I/O LEADS

Validation is I/O home turf. The five layers of validity evidence, fairness analysis, and reliability design are all I/O-owned.

06

Deployment

I/O CONTRIBUTES

Appropriate reliance training design and boundary condition specification are I/O contributions. Deployment mechanics are engineering.

07

Monitoring

I/O LEADS

Ongoing fairness monitoring, automation bias detection, and re-validation trigger definition are I/O-specific expertise.

08

Iteration

I/O LEADS

Re-validation design, evidence update, and Tier classification review are I/O work. Documentation and governance communication too.

Agenda

1. **Setting the Scene**: What is Agentic AI?
2. The Software Development Life Cycle (SDLC) x Psychometrics **Framework**
3. Agentic AI **No-Code** Showcase using *n8n*
4. **Coding** Showcase: *Agentic AI with Python*
5. Q&A + Resources for your learning

© 2026 SHL and its affiliates. All rights reserved.

An HR workflow

Most HR work isn't admin. It's thinking work — done by hand.

Drafting a JD

You stare at last year's template and rewrite half of it.

Researching the role

Skills, levels, comp benchmarks — multiple tabs, lots of copy-paste.

Writing interview questions

You build the same kind of question list, again, for every new req.

1 What is automation?

A computer doing the steps you'd otherwise do by hand.

Trigger
(e.g. "Click run")

→

Action
(e.g. read a list)

It's nothing fancy. It's just steps in a row.

D E M O 1

Triggers & Connectors

Three roles → three rows on screen. No human opens a spreadsheet.

2 What if a step could think?

Same workflow. Now one of the boxes uses an AI to think before passing the result on.

The workflow still runs in order. The AI just does one specific job inside it.

D E M O 2

Adding an LLM step

Imagine the gist of fifty open reqs — in the time it takes to grab coffee.

3 Integrating an Agent

An agent doesn't follow your steps. It chooses them.

Workflow with an AI step

You write the steps:

Trigger → Think → Action.

The AI runs the Think step. The order is fixed.

Agent

You give a goal.

The agent decides which tools to call, in what order, until the goal is met.

Same idea you have when you give a junior teammate a goal and walk away.

D E M O 3

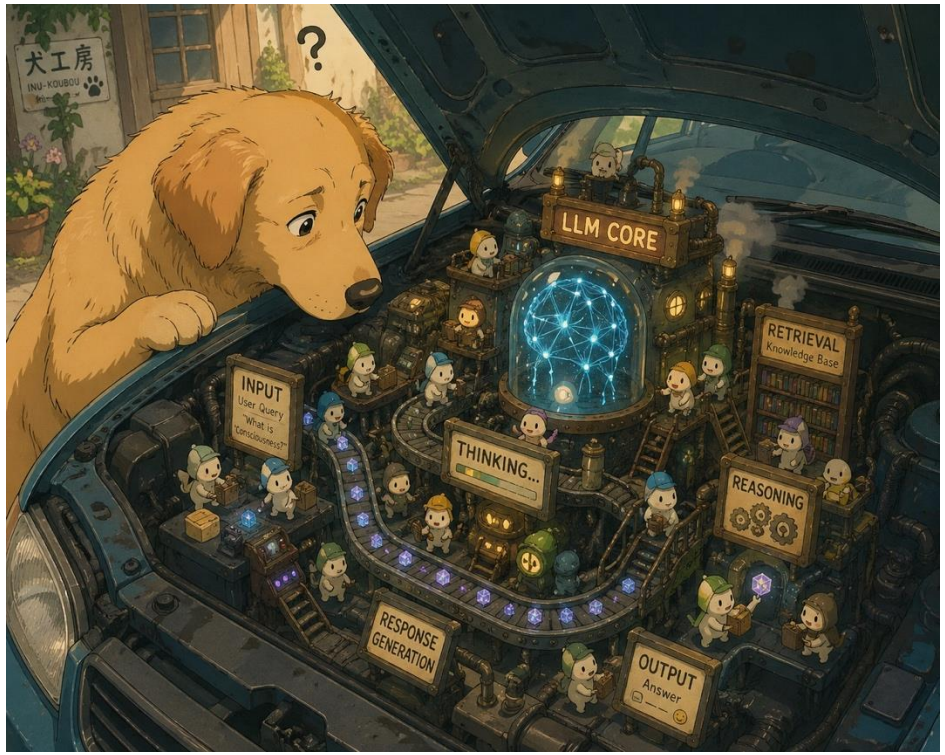
Giving the LLM tools

*Watch what I never told it to do: when to call O*NET, which code to use, when to stop.*

Agenda

1. **Setting the Scene**: What is Agentic AI?
2. The Software Development Life Cycle (SDLC) x Psychometrics **Framework**
3. Agentic AI **No-Code** Showcase using *n8n*
4. **Coding** Showcase: *Agentic AI with Python*
5. Q&A + Resources for your learning

© 2026 SHL and its affiliates. All rights reserved.



We just saw the agentic workflow as boxes and arrows.

Next, let's take a look under the hood.

The agentic workflow is mostly a loop:

- call an LLM
- let it act/ use a tool
- check the result
- continue or stop if the product is good enough

Agent = loop + tools + checks

When to use code vs no-code?

No-code when you're exploring what's possible

- Quickly build & share prototypes
- Fast & visible
- Workflow is linear
- Uses standard tools
- Maintained by non-engineers
- Human review catches most issues
- Learn workflow nuances

Code when you're building tools people rely on

- Need reproducibility & version control
- Require custom validation logic
- Need audit trails/ defensibility
- Uses custom tools
- Integrated with proprietary systems
- Learn input-output pair nuances

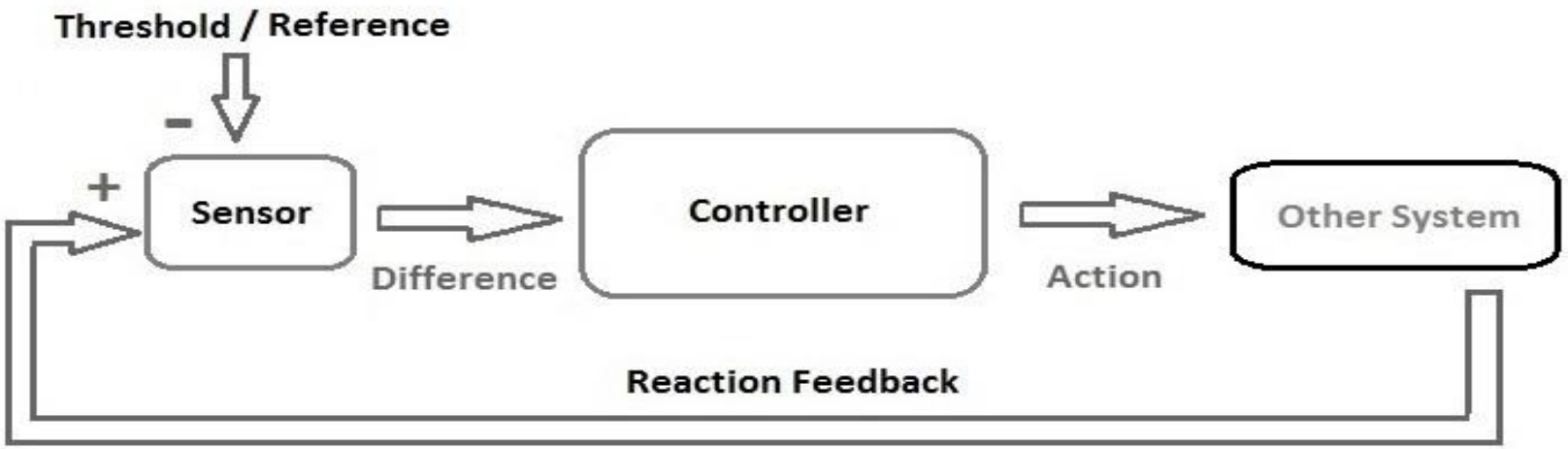
Tldr: both use the same loop logic but no-code shows the workflow & code offers more control

Must haves:

1. Model *(look for a balance of capability x cost x chattiness)*
2. Tools *(search, lookup, code interpreter, etc.)*
3. States/ Memory *(what the system can remember, error handling and recovery)*
4. Evaluators *(check quality of the output)*
5. Human control *(review, approve, override, escalate)*

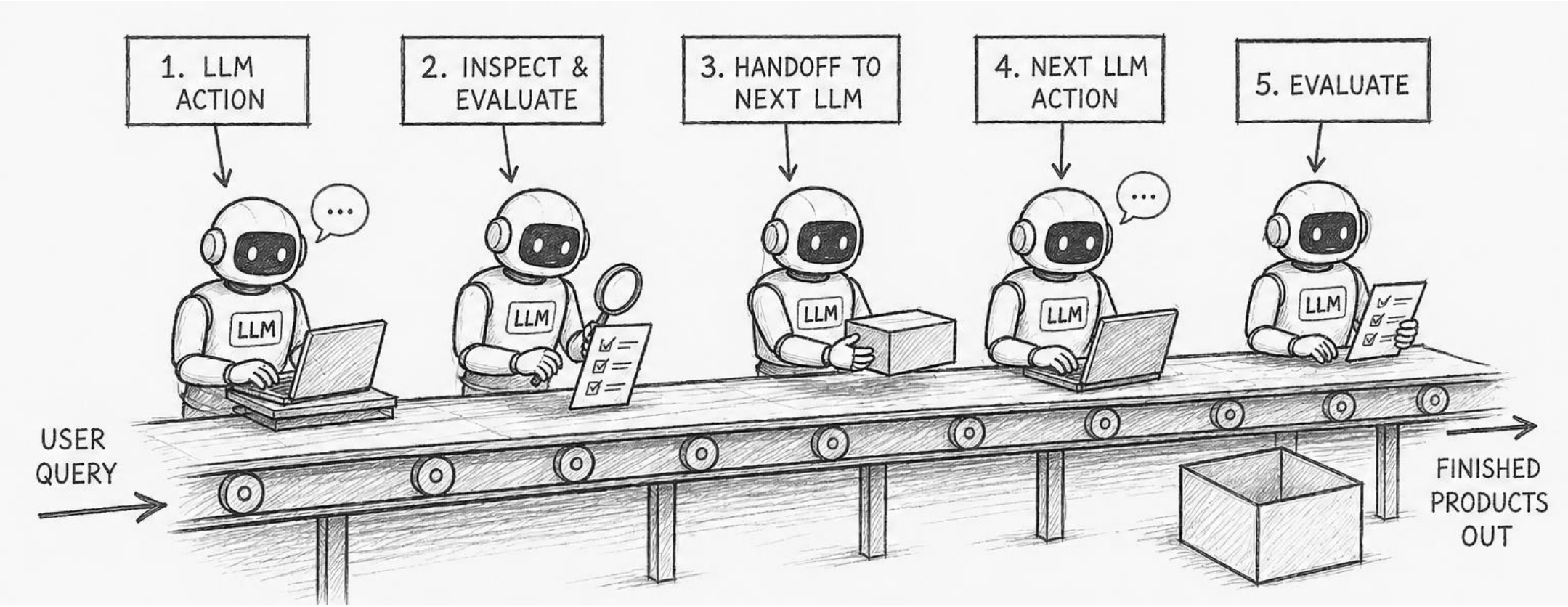
Nice-to-Haves:

Orchestration, scaffolds, multi-agent coordination, automations, scraping, voice, vision, observability/ monitoring, desktop/ MCP integration, tool registries, streaming, caching optimization, cost/ latency management, testing/ sandboxing, user personalization, etc.



A Cybernetic Loop

Control Theory (Powers, 1973)
Self-Regulation (Carver & Scheier, 1998)
Dynamic Control Theory for Goal Pursuit (Vancouver, 2005)


1. LLM ACTION
2. INSPECT & EVALUATE
3. HANDOFF TO NEXT LLM
4. NEXT LLM ACTION
5. EVALUATE
LLM
USER QUERY
FINISHED PRODUCTS OUT

We're building an agent that searches for job tasks and competencies, based on a job description, then builds structured interview questions

```python
while not done: search → evaluate → draft → evaluate → output
```

Loop #1 gathers job info
- Search O*NET
- Search Web

```python
action = decide_next_action(state)
result = search_onet(query) or web_search(query)
state["evidence"].append(result)
```

Job Description

Evaluator #1 checks Agent #1's work against criteria:
- Fails Check? Loop Again
- Passes check? Handoff

```python
review = evaluate_evidence(state)
if not review["sufficient"]:
    continue  # ← back to Robot 1
```

Loop #2 drafts interview guide

```python
guide = generate_interview_guide(state)
state["guide"] = guide
```

Loop #2

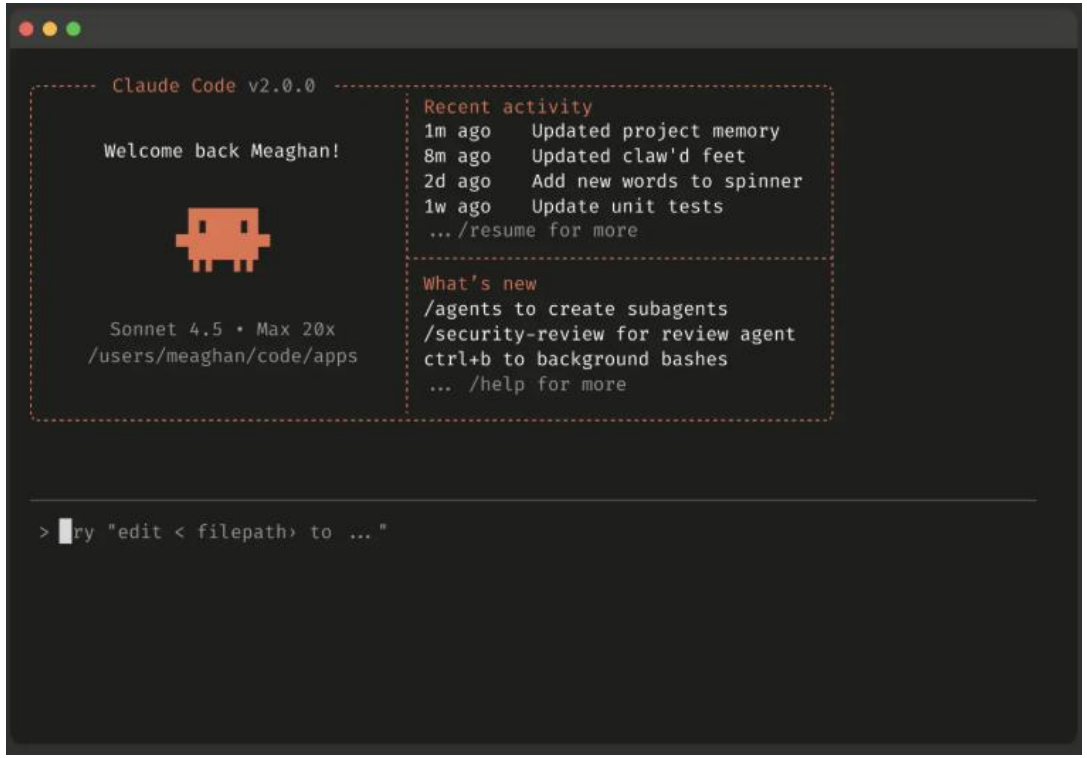
Evaluator #2 checks guide:
Behavioral? Job Relevant? Evidence-backed?
- Fail? Loop Again
- Pass? Output Guide

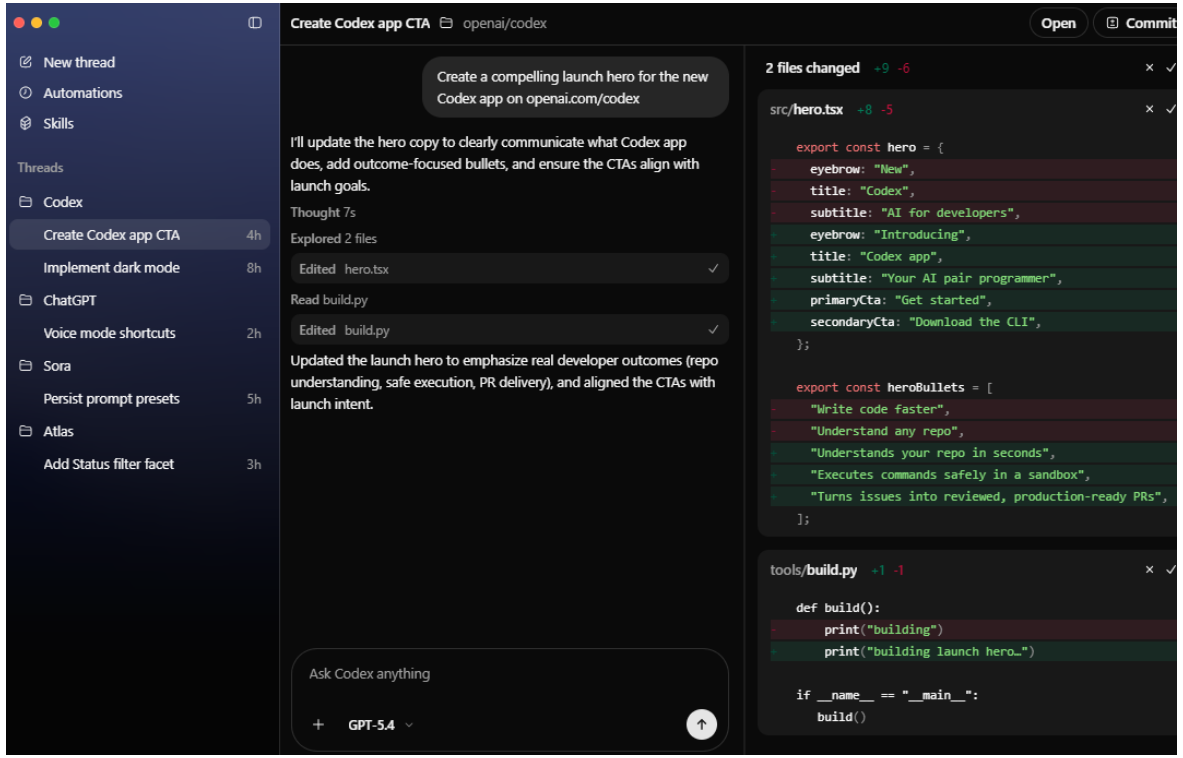
```python
review = evaluate_interview_guide(guide)
if not review["approved"]:
    guide = revise_guide(state)
```

```python
state["done"] = True
```

{demo}

There’s a middle ground between code vs no-code





What You Just Saw

A three-stage agentic workflow in n8n — let's map it to agent concepts

The Workflow

Stage 1 Job Description input → structured competency extraction

Stage 2 Competency mapping to existing Skills Framework

Stage 3 Generation of interview questions aligned to mapped competencies

Agentic Elements Present

- Sequential reasoning across stages
- Tool use (n8n nodes as tools)
- LLM calls with structured outputs

THE KEY QUESTION

If this tool were deployed in your organisation's hiring workflow today — what would you need to know before using it?

The framework gives you a structured way to answer that question. Let's walk through this tool, and see what would still need to happen before responsible deployment.

Mapping the Demo to the Framework

Which steps were addressed, which were skipped, which remain open

01

Governance & Purpose

Partially — use case defined; stakes not classified; HITL not designed

02

Requirements

Partially — functional requirements implicit; no psychometric blueprint or evaluation strategy

03

Design

Yes — workflow architecture designed; prompts written; rubrics not formalised

04

Build

Yes — system built; no version control evident; logging not discussed

05

Validation

Not yet — no content review, reliability testing, or fairness analysis

06

Deployment

Not yet — no pilot plan, HITL mechanism, or user training

07

Monitoring

Not yet — no drift detection or re-validation triggers defined

08

Iteration

Not yet — follows from deployment

Decision Points in Practice

Four questions that arise at Steps 1–2 for this specific tool

What is the Tier?

Is this tool advisory (a recruiter reviews all outputs before acting) or does it filter candidates before human review? The answer changes everything — it may be Tier 1 if outputs feed into shortlisting, even indirectly.

What construct is being measured?

The tool maps JDs to a Skills Framework. What are we actually claiming? That the agent identifies the same competencies a trained I/O analyst would? That claim must be specified before it can be tested.

What does success look like?

How accurate does the competency mapping need to be? 80% agreement with expert analysts? 95%? Who sets that threshold, and what is the consequence of being below it?

Who is in the loop — and how?

What happens when the agent gets it wrong? Is there a structured review mechanism, or do outputs flow straight to hiring managers? HITL design must be decided here, not at deployment.

Agenda

1. **Setting the Scene**: What is Agentic AI?
2. The Software Development Life Cycle (SDLC) x Psychometrics **Framework**
3. Agentic AI **No-Code** Showcase using *n8n*
4. **Coding** Showcase: *Agentic AI with Python*
5. Q&A + Resources for your learning

© 2026 SHL and its affiliates. All rights reserved.

Q&A + Resources

- Q & A
- Slides available on Whova (PDF)
- All materials (slides, code, and supplementary resources) found on **GitHub repo** here:

https://github.com/karimhbadr1/SIOP_2026_Agentic_AI_Master_Tutorial

GitHub Repo

© 2026 SHL and its affiliates. All rights reserved.